Introduction

We implemented the hashing algorithm described in Section 4 of “Modern Hashing Made Simple”. Next, we illustrate an “adversarial” attack on the proposed hashing data structure, which forces insertions to require O(n) operations with probability 1. Both implementations available in the following Github Repository

Implementation of Hashing Data Structure

To recap, the data structures in Sections 4 and 5 of the paper are summarized below:

	Insertion	Deletion	Query
Section 4	O(1) in expectation	O(1)	O(1)
Section 5	Amortized O(1) w.h.p	O(1)	O(1)

In particular, the O(1) w.h.p. guarantee in Section 5 is achieved by caching incoming operations and executing them in batches, analyzing the amortized time complexity over the entire batch, rather than processing each operation immediately upon arrival as in Section 4. We further assume that the hash table is sufficiently large and does not require resizing. Due to time constraints, we implemented the algorithm in Section 4.

“Adversarial” Attack on Hashing Data Structure

We assume the role of an adversary aiming to degrade the performance of the hash table, such that queries can no longer be performed in constant time. The adversary is assumed to have full knowledge of the system, including the exact hash function and the seed of the pseudorandom number generator (PRNG). With this knowledge, the adversary can craft inputs that are mapped to the same randomly chosen bucket. Our empirical evaluation demonstrates that such an attack can increase the average query time by approximately 13×.

In practice, if an adversary can consistently map all queries to a few selected buckets, they will be able to force the hash table to undergo many resizing operations. This would further degrade the overall performance of the hash table, compounding the impact of the attack resulting in O(n) operations needed for insertions (for both data structures in Section 4 and 5).

Conclusion

To the best of our knowledge, we are the first to provide a code implementation of the techniques described in this paper. Next, an “adversarial” attack was proposed on the hashing data structure, which successfully degrades the performance of the hash table to O(n) with probability 1. Our empirical results show this results in a significant increase in time taken to query the hash table, and in practice, it would be even more severe when hash table resizing is triggered. This underscores the importance of using a secure, collision-resistant hash function.